Attention 加速器 IP 设计与验证

项目名称：Attention 加速器 IP 设计与验证

所选题目：A 题（基础版，S = 8，d = 4，Batch/Head = 1/1）

作者 / 日期：张仕涵/2026.10.7

工具版本：仿真器 Icarus Verilog（iverilog / vvp，-g2012），波形查看 GTKWave；参考模型 Python 3.14

1 项目背景与公式理解

1.1 项目背景

这次实践项目的题目是把 Attention 算法的一小部分写成真正能综合的硬件。Attention 本身做的事并不复杂：当前这个 Token 拿着自己的 Q 去和历史上每个 Token 的 K 比对，看它们有多像，像得越多，就从对应的 V 里取越多内容出来。基于这个定位，我把本次设计的目标拆成了三件事。第一件是正确，也就是要按照题目的公式把 Score、权重、L、ACC 和最终输出 O 一步一步算对，并且用 Python 的参考模型逐项对齐。第二件是清楚，数据通路要分阶段，每个阶段干什么、数据从哪个寄存器来、算完写回哪里，而不是把所有运算塞进一个大的组合逻辑里。第三件是可综合、可解释，只能使用寄存器、Buffer、乘法器、加法器、累加器、查找表和有限状态机这些基本单元，模块的职责和数据流向都要是可以解释的。

1.2 核心公式推导

整个计算分五步。第一步算 Score，把 Q 和每一个 K 逐维相乘再相加：

$$score_j = Q \cdot K_j = \sum Q[k] \times K_j[k]$$

第二步做数值稳定化。如果把 Score 直接丢进 exp，Score 稍大一点指数就会冲得很高，所以先找出 8 个 Score 中的最大值，再让每个 Score 去减它，这样指数输入永远小于等于零，结果落在 0 到 1 之间：

$$weight_j = exp(score_j)$$

第三步把权重加起来得到分母 L，第四步把权重乘上对应的整个 V 向量并累加得到分子 ACC，第五步归一化，用 ACC 除以 L 得到输出：

$$L = \sum weight_j \qquad ACC = \sum weight_j \times V_j \qquad O = ACC / L$$

2 整体数据流与硬件架构

2.1 顶层模块

整个设计分成两个模块：顶层 attention_ip 和指数查找表 exp_lut。顶层按题目给的接口定义端口，输入有时钟 clk、低有效复位 rst_n、启动信号 start，三个 8 位有符号数据 q_in、k_in、v_in，以及两个有效标志 q_valid、kv_valid；输出有 data_ready、8 位有符号的 out_data、out_valid 和 done。exp_lut 是纯组合逻辑，输入是 18 位有符号的 exp_x，输出是 5 位无符号的权重。

存储用了六个寄存器阵列：q_buf 存 4 个 Q 元素，k_buf 和 v_buf 各存 8 乘 4 个 K 和 V，score_buf 存 8 个 Score，weight_buf 存 8 个权重，acc_buf 存 4 个维度的 ACC。运算单元只有两个乘法器、一个除法器、一个比较器和一张查找表。

2.2 数据流说明

输入阶段分两步。先收 Q：状态机进入 LOAD_Q 后，每拍从 q_in 收一个数写进 q_buf，计数器 q_cnt 加一，收到第 4 个（q_cnt 等于 3）时把计数器清零并转入下一个状态。再收 K 和 V：LOAD_KV 状态下 k_in 和 v_in 同时有效，每拍把这两个数分别写进 k_buf[token_cnt][dim_cnt] 和 v_buf[token_cnt][dim_cnt]，dim_cnt 从 0 数到 3 表示一个 Token 的 4 个维度装完了，然后 token_cnt 加一换下一个 Token，直到第 8 个 Token 的第 4 维装完，一共 32 拍。

Score 阶段进入 DOT 状态，只有一根累加器和一根维度计数器。每拍做一次 q_buf[dim_cnt] 乘 k_buf[token_cnt][dim_cnt] 并累加到 score_acc 上，前三维先攒着，到第四维把这一项也加上、结果锁进 score_buf[token_cnt]，然后累加器清零、维度计数器清零、Token 计数器加一，开始下一个 Token。8 个 Token 走完是 32 拍。

权重阶段都在 EXP 状态里，用 exp_phase 分成两段。第一段求最大值：先把 score_buf[0] 当作当前最大，每拍和一个新的 Score 比较，遇到更大的就更新 max_score；比较到第八个时这一段结束，同时把查表的第一个输入（score_buf[0] 减 max_score）算好，并把 exp_phase 置一。第二段查表：每拍把 exp_lut 的输出写进 weight_buf 的当前位置，写当前这一个的同时把下一个查表输入（score_buf[j+1] 减 max_score）提前算好放到 exp_x 上，这样下一拍的查找表输出是稳定的，查表这 8 拍不会空等。因为 exp_lut 是组合逻辑，"提前一拍算输入"正好把它的延迟藏起来。

累加阶段在 UPDATE 状态，用一位标志 judge_L 分两遍跑同一个累加器。第一遍 每拍把一个权重加到 L 上，8 拍累加完得到分母，这一遍的最后把 acc_acc 清零、judge_L 清成 0。第二遍算分子，按维度组织：每个维度把 8 个 Token 的权重乘 V 累加进 acc_acc，累加完写进 acc_buf 对应位置，再换下一个维度，4 个维度一共 32 拍。

归一化和输出在 NORM 与 OUTPUT 两个状态。NORM 每拍处理一个维度，算出 0 后拉高 out_valid 并把结果放到 out_data 上，4 拍输出 4 个元素；OUTPUT 只把 done 拉高一拍并回到 IDLE，随时可以接下一个任务。

3 模块划分与关键设计

3.1 子模块划分

题目大纲里列了七个功能块，我把它们归成了两个 Verilog 模块：attention_ip 和 exp_lut。其中除"指数查找表"独立成 exp_lut 外，其余功能块都写在 attention_ip 内部。输入接口与寄存器对应 LOAD_Q、LOAD_KV 两个状态和 q_buf、k_buf、v_buf 三组寄存器；点积单元对应 DOT 状态里的乘法器加 score_acc 累加器；最大值检索对应 EXP 第一段和比较器，每个时钟周期比一个、串行更新 max_score，8 个周期出结果，指数查找表对应独立的 exp_lut 模块；权重与 ACC 累加对应 UPDATE 的两遍累加；归一化对应 NORM 里的有符号除法；输出接口与状态机则是 data_ready 的组合译码、out_valid 与 done 的脉冲产生，以及状态寄存器本身。

3.2 指数查找表设计

exp_lut 是这次设计里需要自己"造"的运算单元，输入是 exp_x，也就是 score 减 max_score 的结果，18 位有符号，正常路径下一定小于等于零；输出是 5 位无符号的权重。

权重的定点格式是这样推出来的：题目要求小数固定保留 4 位，量化步长就是 1/16，浮点值 1.0 对应的编码是 16；又因为 exp(x) 在 x 小于等于零时最大就是 1，整数部分只需要 1 位。1 位整数加 4 位小数等于 5 位，所以输出端口取 5 位宽，取值范围 0 到 31，实际用到的是 0 到 16。表的内容就是把 exp(x) 乘 16 再四舍五入：x 等

于 0 时是 16，等于 -1 时是 6，等于 -2 时是 2，等于 -3 时是 1，再往下乘 16 都不足 0.5，直接是 0。所以代码里只写了这四档，其余由 default 统一输出 0。

x	e^x	×16	四舍五入	代码里写的
0	1.0000	16.000	**16**	`5'd16`
-1	0.36788	5.886	**6**	`5'd6`
-2	0.13534	2.165	**2**	`5'd2`
-3	0.04979	0.797	**1**	`5'd1`
-4	0.01832	0.293	0	落到 `default`
-5 及以下	≤0.00674	≤0.108	0	落到 `default`

3.3 位宽设计

位宽是一边推导一边定下来的。输入 Q、K、V 按题目是 8 位有符号，范围 -128 到 127。Score 通路上，两个最大正 8 位数相乘最大是 127 乘 127 等于 16129，4 项累加后接近 6.5 万，而 16 位有符号最大只到 32767，明显不够，所以 score_acc、score_buf、max_score 和 exp_x 统一取 18 位，能表示正负十三万多。ACC 通路上，权重最大 16、V 的绝对值最大 128，单项乘积不超过 16×128=2048，8 个 Token 累加不超过 16384 的量级，16 位有符号装得下，所以 acc_acc 和 acc_buf 取 16 位。L 是 8 个权重相加，最大 128，取 8 位无符号。输出按题目要求是 8 位有符号。

4 状态机设计

4.1 状态机

状态机一共用了八个状态，分别是 IDLE、LOAD_Q、LOAD_KV、DOT、EXP、UPDATE、NORM 和 OUTPUT，用四位寄存器编码。IDLE 里采到 start 就进 LOAD_Q；LOAD_Q 连着收到四个 q_valid 之后进 LOAD_KV；LOAD_KV 收满 32 组 K/V 之后进 DOT；DOT 把八个 Score 全部写进 score_buf 后进 EXP；EXP 查完八个权重后进 UPDATE；UPDATE 把分母和四个维度的分子都算完后进 NORM；NORM 输出完四个元素后进 OUTPUT；OUTPUT 拉高 done 并回到 IDLE。

写状态机的时候我特意把 done 和 out_valid 的默认清零放在 always 块最前面，每个周期先把它们清掉，只有真的在输出或者结束时才置位。因为如果放到各个状态里分别清零，很容易出现某个分支忘了清、脉冲变成一直拉高的情况，下游就会重复采样。

4.2 时序统计与参考时序对比

各阶段占用的时钟周期如下表。一次完整任务一共 131 个周期；

阶段	状态	周期数	说明
装载 Q	LOAD_Q	4	每拍 1 个元素，d = 4
装载 K/V	LOAD_KV	32	每拍 1 组 (K, V)，S×d = 32
Score 计算	DOT	32	每拍 1 次乘加，每 4 拍出 1 个 Score

求最大值（MAX）	EXP 段 1	8	串行比较，每拍 1 个 Score
查表求权重	EXP 段 2	8	每拍查 1 个，同时预取下一个输入
累加分母 L	UPDATE 第 1 遍	8	与分子复用同一个累加器
累加分子 ACC	UPDATE 第 2 遍	32	每 8 拍出 1 个维度的 ACC
归一化	NORM	4	每拍输出 1 个元素
结束	OUTPUT	1	done 脉冲，回到 IDLE
合计（含启动等待 2 拍）	131	1 + 2d + 3Sd + 2S + 1	

5 验证方法与结果

5.1 Golden Model

按题目要求参考模型用 Python 写，所以验证的基准是 golden_model.py。它的流程和 RTL 完全一样：先随机生成一组 Q、K、V（范围 -8 到 8，8 个 Token 各 4 维），然后按 score、x_max、weight、L、ACC、O 的顺序算一遍，最后打印结果，同时把这一组数据打印成 Verilog 的 send_q / send_kv 语句，方便直接填进测试平台。归一化用的是 int((acc + L//2) / L) 和 acc 为负时的 int((acc - L//2) / L)，和 Verilog 里 (ACC ± L/2)/L 的语义相同，都是向零取整。

5.2 Testbench 结构

测试平台有两个。tb_exp_lut.v 是最小的单元测试，只测查找表：把 x 依次设成 0、-1、-2、-3、-4 ，每个值等 10 纳秒后打印输出，确认四档数值和截断边界。tb_attention_ip.v 是定向测试，里面有两个任务 send_q 和 send_kv，都是把 valid 拉高一拍、下一拍拉低，和 RTL 每拍采一个元素的节奏对齐；激励送完后把波形 dump 到 attention.vcd，并打印 acc_buf 的四个值，再拿它和 Python 参考模型对一遍。

5.3 仿真波形与逐周期说明

波形用 GTKWave 打开，主要看 clk、rst_n、start、state、data_ready、q_valid、kv_valid、out_data、out_valid、done 这几组信号。tb_attention_ip.v 里发出去的激励是 Q 等于 [−7, 3, 0, −5]，接着是 8 组 K/V、一共 32 对，开头几对是 (8, −1)、(8, −2)、(7, 1)、(5, −3)，最后几对是 (−4, −4)、(1, 4)、(4, 6)、(4, 0)、(−6, −5)、(0, 4)、(−4, −1)、(−3, 7)。对着这组激励看波形，一开始 state 停在 IDLE 等 start；start 被采到后进 LOAD_Q，data_ready 从这一拍起变高，连续 4 拍把 Q 装完，波形上能看到 q_buf 依次变成 −7、3、0、−5；接着 32 拍装 K/V，每拍 dim_cnt 和 token_cnt 都在动，两个计数器一格一格走过去，每 4 拍换一个 Token；装完立刻进 DOT，从这时起 data_ready 变低，上游不再送数据；DOT 阶段同样是每 4 拍往 score_buf 里写一个数，8 个 Token 走完 32 拍，score_buf 依次是 −57、67、87、−102、−45、96、11、57；进入 EXP 后先是 8 拍比较，max_score 在比到第 6 个 Token 时更新成 96，后面几个 Score 都比它小、不再变化；接着 8 拍查表，weight_buf 每拍多一个值，只有第 6 个 Token 拿到 16（因为它的 Score 减最大值正好等于 0），其余全是 0；UPDATE 的前 8 拍 L 一拍一次累加，最后等于 16，后 32 拍每 8 拍往 acc_buf 写一个维度，依次是 80、−128、48、112；NORM 的 4 拍里 out_valid 一直为高，out_data 依次是 5、−8、3、7；最后 done 拉高一拍，state 回到 IDLE。整段从 start 到 done 一共 131 拍。

6 遇到的问题与改进方向

6.1 调试中遇到的主要问题

第一个问题是工程配置。最初在 Vivado 里建工程，只把 attention_ip.v 和 tb_attention_ip.v 加了进去，忘了 exp_lut.v，结果仿真点了半天波形窗口一直是空的。翻 Tcl Console 才看到报错说 exp_lut 这个模块没有声明——attention_ip 里例化了它，但工程里没有它的代码，细化阶段就失败了。补进 exp_lut.v 之后又出现第二个问题：testbench 没有被当成顶层，Design Sources 里混着测试文件，要把它移到 Simulation Sources 并设成 top，但是可能由于环境配置问题，波形还是没有显示出来，后来只能换了工具进行仿真。

第二个问题是工具链入门。一开始既不清楚 iverilog 和 GTKWave 是怎么配合的，也不知道命令行该怎么敲，还试过用 PyCharm 去打开 Verilog 文件。后来按"iverilog 编译出 vvp 文件、vvp 跑仿真、gtkwave 看 vcd 波形"这三步走顺了，也理解了编译和仿真是两件事：编译只管语法，仿真才产生结果，波形只是仿真的副产品。

第三个问题也是在汇报前临时出现的问题，我发现用 golden_model 算出来的随机 Q 输入有时候会丢失第一个 Q 的波形，询问 AI，它告诉我可能是 gtkwave 的像素过低从而忽略了第一个值，可能也是因为我在写 golden_model 的时候没有限定好范围导致这个问题发生。

6.2 解决方式

回头看这些问题都不复杂，还是要多查资料，多尝试多试错，才能纠正错误。

6.3 改进方向

最想改的是权重精度。现在小数只有 4 位，exp(-3) 的相对误差较大，x 小于等于 -4 直接被截断成 0，随机数据下经常只有一个权重非零，输出退化成某个 V 的复制。根据查找的资料，如果把权重扩到 7 到 8 位、把表补到 x 等于 -4 甚至 -6，Softmax 就能在更多 Token 之间分配权重，输出会更接近浮点结果。另一个思路是在点积结果上做一次算术右移再减最大值，相当于给 Softmax 降一点"温度"，不加宽查找表也能提高有效分辨率。